PHASE 1 — Project Setup, Docker, Database Schema & Authentication

PRSECURITY HRMS

IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.
After each file write "--- FILE COMPLETE ---" then start the next file immediately.
If you are about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"
This is Phase 1 of 8. Later phases will add more features on top of this foundation.

PROJECT OVERVIEW

You are building an internal HRMS (Human Resource Management System) web app for: **PRSECURITY CONSULTANCY & SERVICES** — a cybersecurity company in Surat, India.
~30 employees. 4 roles: ADMIN, HR, EMPLOYEE, INTERN. All amounts in INR ₹. Data must persist permanently in PostgreSQL.

TECH STACK

Frontend

React 18 + Vite + TypeScript
Tailwind CSS + shadcn/ui
React Router v6
React Hook Form + Zod
Axios
Zustand (global state)
Socket.io-client

Backend

Node.js + Express + TypeScript
Prisma ORM + PostgreSQL
JWT (access token 15min + refresh token 7 days in httpOnly cookie)
bcryptjs (saltRounds: 12)
cors, helmet, express-rate-limit, dotenv

FOLDER STRUCTURE TO CREATE

```
```

```
orsecurity-hrms/
├── frontend/
│   ├── src/
│   │   ├── api/
│   │   │   └── axios.ts
│   │   ├── store/
│   │   │   └── authStore.ts
│   │   ├── types/
│   │   │   └── index.ts
│   │   ├── hooks/
│   │   │   └── useAuth.ts
│   │   ├── components/
│   │   │   └── layout/
│   │   │       ├── Sidebar.tsx
│   │   │       ├── Topbar.tsx
│   │   │       └── MainLayout.tsx
│   │   ├── routes/
│   │   │   └── ProtectedRoute.tsx
│   │   ├── pages/
│   │   │   └── auth/
│   │   │       ├── LoginPage.tsx
│   │   │       ├── ForgotPasswordPage.tsx
│   │   │       └── ResetPasswordPage.tsx
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── index.html
│   ├── vite.config.ts
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   └── .env
│
├── backend/
│   ├── src/
│   │   ├── app.ts
│   │   ├── middleware/
│   │   │   ├── auth.middleware.ts
│   │   │   ├── roleGuard.middleware.ts
│   │   │   └── errorHandler.middleware.ts
│   │   ├── routes/
│   │   │   └── auth.routes.ts
│   │   ├── controllers/
│   │   │   └── auth.controller.ts
│   │   ├── services/
│   │   │   └── auth.service.ts
```

```
│   │   └── utils/
│   │       ├── jwt.utils.ts
│   │       └── response.utils.ts
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── seed.ts
│   ├── .env
│   └── tsconfig.json
│
├── docker-compose.yml
└── README.md
```

FILE 1: docker-compose.yml

PostgreSQL 15 + pgAdmin. Include named volume for data persistence. PostgreSQL on port 5432, pgAdmin on port 5050. DB name: prsecurity_hrms, user: prsecurity, password: prsecurity@123

FILE 2: backend/prisma/schema.prisma

Build the COMPLETE Prisma schema with ALL these models:

Enums

```
Role: ADMIN | HR | EMPLOYEE | INTERN
EmploymentType: FULL_TIME | PART_TIME | INTERN | CONTRACT
UserStatus: ACTIVE | INACTIVE | ON_LEAVE | TERMINATED
```

User model

```
id (uuid), employeeId (unique, String), firstName, lastName,
email (unique), passwordHash, role (Role), phone (optional),
profilePicture (optional), dateOfBirth (optional DateTime),
dateOfJoining (DateTime default now), designation (optional),
employmentType (EmploymentType default FULL_TIME),
status (UserStatus default ACTIVE),
address (optional), emergencyContactName (optional),
emergencyContactPhone (optional), bankAccountNumber (optional),
bankIFSC (optional), bankName (optional), panNumber (optional),
refreshToken (optional), passwordResetToken (optional),
passwordResetExpiry (optional DateTime),
createdAt (default now), updatedAt (updatedAt)

Relations: SalaryStructure (one-to-one), Attendance (one-to-many),
LeaveBalance (one-to-many), LeaveRequest (one-to-many),
Payroll (one-to-many), Notification (one-to-many)
```

SalaryStructure model

```
id (uuid), userId (unique FK → User), basicSalary (Float),
hra (Float default 0), travelAllowance (Float default 0),
medicalAllowance (Float default 0), otherAllowances (Float default 0),
pfEmployeePercent (Float default 12), pfEmployerPercent (Float default 12),
esiEmployeePercent (Float default 0.75), esiEmployerPercent (Float default 3.25),
tdsPercent (Float default 0), createdAt, updatedAt
```

Notification model

```
id (uuid), userId (FK → User), title, message,
type (String), isRead (Boolean default false),
link (optional String), createdAt (default now)
```

HolidayCalendar model

```
id (uuid), name, date (DateTime), type (String), year (Int), createdAt
```

Also include placeholder models with just id+createdAt for: Attendance, LeaveType, LeaveBalance, LeaveRequest, Payroll, Project, Client, Task, JobPosting, Applicant, PerformanceGoal, PerformanceReview, Asset, ExpenseClaim, Announcement, HelpdeskTicket (These will be fully expanded in later phases)

FILE 3: backend/prisma/seed.ts

Seed with:
Admin account: email=admin@prsecurity.in, password=Admin@1234, role=ADMIN, firstName=Super, lastName=Admin, employeeId=PRS-000
HR account: email=hr@prsecurity.in, password=Hr@1234, role=HR, firstName=HR, lastName=Manager, employeeId=PRS-001
SalaryStructure for both: basicSalary=50000
10 Indian national holidays for current year in HolidayCalendar

FILE 4: backend/.env

```
```

```
DATABASE_URL="postgresql://prsecurity:prsecurity@123@localhost:5432/prsecurity_hrms"
JWT_SECRET="prsecurity-jwt-secret-key-minimum-32-characters-long"
JWT_REFRESH_SECRET="prsecurity-refresh-secret-key-minimum-32-characters"
JWT_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"
PORT=5000
CLIENT_URL="http://localhost:5173"
NODE_ENV="development"
SMTP_HOST="smtp.gmail.com"
SMTP_PORT=587
SMTP_USER="noreply@prsecurity.in"
SMTP_PASS="your-app-password"
```

FILE 5: backend/tsconfig.json

Strict TypeScript config for Node.js with ES2020 target, CommonJS module, outDir=dist, rootDir=src.

FILE 6: backend/src/utils/jwt.utils.ts

Functions:

`generateAccessToken(payload: { id, role })` → signed JWT

`generateRefreshToken(payload: { id })` → signed JWT

`verifyAccessToken(token)` → decoded payload or throws

`verifyRefreshToken(token)` → decoded payload or throws

FILE 7: backend/src/utils/response.utils.ts

Functions:

`successResponse(res, data, message, statusCode=200)`

`errorResponse(res, message, statusCode=400, errors?)`

Both return JSON: `{ success: bool, message: string, data?: any, errors?: any }`

FILE 8: backend/src/middleware/auth.middleware.ts

Express middleware that:
Reads Authorization header Bearer token
Verifies JWT using verifyAccessToken
Attaches decoded user to req.user
Returns 401 if missing or invalid

FILE 9: backend/src/middleware/roleGuard.middleware.ts

Factory function `roleGuard(roles: Role[])` that returns Express middleware checking req.user.role is in allowed roles. Returns 403 if not.

FILE 10: backend/src/middleware/errorHandler.middleware.ts

Global Express error handler. Catches Prisma errors (P2002 unique violation → 409, P2025 not found → 404), Zod errors → 400 with field-level errors, JWT errors → 401, all others → 500.

FILE 11: backend/src/services/auth.service.ts

Functions:

`login(email, password)` → validates credentials, returns user + tokens

`refreshTokens(refreshToken)` → validates refresh token stored in DB, returns new tokens

`logout(userId)` → clears refreshToken in DB

`getMe(userId)` → returns user without passwordHash

`changePassword(userId, oldPassword, newPassword)` → validates + updates

`forgotPassword(email)` → generates reset token (crypto.randomBytes), stores hash+expiry in DB, returns token (caller sends email)

`resetPassword(token, newPassword)` → validates token not expired, updates password, clears token

FILE 12: backend/src/controllers/auth.controller.ts

Express controllers calling auth.service for: POST /login, POST /refresh, POST /logout, GET /me, POST /change-password, POST /forgot-password, POST /reset-password/:token

FILE 13: backend/src/routes/auth.routes.ts

Express router mounting all auth controller methods. Apply rate limiting (5 requests/15min) on /login and /forgot-password routes.

FILE 14: backend/src/app.ts

Express app setup:
helmet(), cors({ origin: CLIENT_URL, credentials: true }), express.json(), cookieParser()
Rate limiting (100/15min global)
Mount /api/auth router
Mount errorHandler middleware last
Export app and a startServer function that connects Prisma then listens on PORT
Initialize Socket.io on the HTTP server, store io instance for use in controllers

FILE 15: frontend/.env

```
VITE_API_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
```

FILE 16: frontend/vite.config.ts

Standard Vite + React config with path alias `@` → `./src`

FILE 17: frontend/tailwind.config.ts

Tailwind config with:
darkMode: 'class'
Content paths for src
Extended theme colors:
primary: `#1E3A5F`
accent: `#00C2FF`
background: { light: `#F8FAFC`, dark: `#0F172A` }
card: { light: `#FFFFFF`, dark: `#1E293B` }

FILE 18: frontend/tsconfig.json

Strict TypeScript config with path alias @/* → src/*

FILE 19: frontend/src/types/index.ts

TypeScript interfaces for ALL entities:

typescript

```ts
export type Role = 'ADMIN' | 'HR' | 'EMPLOYEE' | 'INTERN'
export type UserStatus = 'ACTIVE' | 'INACTIVE' | 'ON_LEAVE' | 'TERMINATED'

export interface User {
  id: string
  employeeId: string
  firstName: string
  lastName: string
  email: string
  role: Role
  phone?: string
  profilePicture?: string
  dateOfBirth?: string
  dateOfJoining: string
  designation?: string
  employmentType: string
  status: UserStatus
  address?: string
  emergencyContactName?: string
  emergencyContactPhone?: string
  bankAccountNumber?: string
  bankIFSC?: string
  bankName?: string
  panNumber?: string
  salaryStructure?: SalaryStructure
  createdAt: string
  updatedAt: string
}

export interface SalaryStructure {
  id: string
  userId: string
  basicSalary: number
  hra: number
  travelAllowance: number
  medicalAllowance: number
  otherAllowances: number
  pfEmployeePercent: number
  pfEmployerPercent: number
  esiEmployeePercent: number
  esiEmployerPercent: number
  tdsPercent: number
}
```

```
export interface Notification {
 id: string
 userId: string
 title: string
 message: string
 type: string
 isRead: boolean
 link?: string
 createdAt: string
}

export interface ApiResponse<T> {
 success: boolean
 message: string
 data?: T
 errors?: any
}

export interface PaginatedResponse<T> {
 data: T[]
 total: number
 page: number
 limit: number
 totalPages: number
}
```

FILE 20: frontend/src/api/axios.ts

Axios instance with:
baseURL from VITE_API_URL
withCredentials: true
Request interceptor: attach Authorization: Bearer {accessToken from store}
Response interceptor: on 401, call /auth/refresh to get new token, retry original request once. On second 401, clear auth store and redirect to /login.

FILE 21: frontend/src/store/authStore.ts

Zustand store with:

```
typescript
```

```
interface AuthStore {
 user: User | null
 accessToken: string | null
 isAuthenticated: boolean
 setAuth: (user: User, token: string) => void
 setToken: (token: string) => void
 clearAuth: () => void
}
```

Persist to localStorage.

FILE 22: frontend/src/hooks/useAuth.ts

Custom hook returning { user, isAuthenticated, role, isAdmin, isHR, isEmployee, isIntern, logout }

FILE 23: frontend/src/api/auth.api.ts

Functions calling the backend:

`login(email, password)` → POST /auth/login

`logout()` → POST /auth/logout

`getMe()` → GET /auth/me

`changePassword(oldPassword, newPassword)` → POST /auth/change-password

`forgotPassword(email)` → POST /auth/forgot-password

`resetPassword(token, newPassword)` → POST /auth/reset-password/:token

`refreshToken()` → POST /auth/refresh

FILE 24: frontend/src/pages/auth/LoginPage.tsx

Full login page UI:

Centered card on navy (`#1E3A5F`) gradient background

PRSECURITY logo/name at top with cyan accent

Email + Password fields using React Hook Form + Zod validation

Show/hide password toggle

"Forgot Password?" link

Login button with loading spinner

On success: call setAuth, redirect to /dashboard

Error toast on failed login

Mobile responsive

FILE 25: frontend/src/pages/auth/ForgotPasswordPage.tsx

Forgot password form: email input, submit sends forgot password API, shows success message with instructions.

FILE 26: frontend/src/pages/auth/ResetPasswordPage.tsx

Reset password form: reads token from URL params, new password + confirm password fields with Zod validation, submits to API, redirects to login on success.

FILE 27: frontend/src/routes/ProtectedRoute.tsx

Route wrapper that:
Checks isAuthenticated from store
If not authenticated → redirect to /login
Accepts optional `allowedRoles: Role[]` prop, redirects to /unauthorized if role not allowed
Shows loading state while checking auth

FILE 28: frontend/src/components/layout/Sidebar.tsx

Full sidebar component:
Fixed left, 260px wide
Collapsible to 64px icon-only mode (toggle button)
On mobile: hidden by default, slides in as drawer
PRSECURITY logo + company name at top
Navigation items with Lucide React icons:
Dashboard (all roles)
Employees (HR/Admin only)
Attendance (all roles)
Leave (all roles)
Payroll (all roles)
Tasks (all roles)
Projects (all roles)
Clients (HR/Admin only)
Recruitment (HR/Admin only)
Performance (all roles)
Assets (HR/Admin only)
Expenses (all roles)
Announcements (all roles)
Helpdesk (all roles)
Reports (HR/Admin only)
Settings (Admin only)
Active item highlighted with accent color
Role-based item visibility
Bottom: user avatar + name + logout button

FILE 29: frontend/src/components/layout/Topbar.tsx

Top bar component:
64px height
Left: hamburger menu (mobile), current page title
Right: Dark mode toggle, Notification bell with unread count badge, User avatar with dropdown (Profile, Change Password, Logout)

FILE 30: frontend/src/components/layout/MainLayout.tsx

Layout wrapper combining Sidebar + Topbar + main content area. Handles sidebar open/close state. Passes page title via outlet context or prop.

FILE 31: frontend/src/App.tsx

React Router setup with ALL routes (pages will be added in later phases — use placeholder components for now):

```
/ → redirect to /dashboard
/login → LoginPage (public)
/forgot-password → ForgotPasswordPage (public)
/reset-password/:token → ResetPasswordPage (public)
/unauthorized → simple "Access Denied" page

Protected routes (wrapped in MainLayout + ProtectedRoute):
/dashboard → DashboardPage (placeholder)
/employees → EmployeesPage (placeholder) [HR/Admin]
/employees/new → NewEmployeePage (placeholder) [HR/Admin]
/employees/:id → EmployeeDetailPage (placeholder)
/attendance → AttendancePage (placeholder)
/attendance/team → TeamAttendancePage (placeholder) [HR/Admin]
/leave → LeavePage (placeholder)
/leave/team → TeamLeavePage (placeholder) [HR/Admin]
/payroll → PayrollPage (placeholder)
/tasks → TasksPage (placeholder)
/projects → ProjectsPage (placeholder)
/clients → ClientsPage (placeholder) [HR/Admin]
/recruitment → RecruitmentPage (placeholder) [HR/Admin]
/performance → PerformancePage (placeholder)
/assets → AssetsPage (placeholder) [HR/Admin]
/expenses → ExpensesPage (placeholder)
/announcements → AnnouncementsPage (placeholder)
/helpdesk → HelpdeskPage (placeholder)
/reports → ReportsPage (placeholder) [HR/Admin]
/settings → SettingsPage (placeholder) [Admin]
/profile → ProfilePage (placeholder)
```

FILE 32: frontend/src/main.tsx

React entry point with Toaster (shadcn/ui) setup.

FILE 33: README.md (Phase 1 section)

Include:
Prerequisites
Docker setup command
Backend setup: npm install, prisma migrate dev, prisma db seed, npm run dev
Frontend setup: npm install, npm run dev
Default credentials
What is built in this phase

AFTER ALL FILES ARE COMPLETE

Write this exactly: "✅ PHASE 1 COMPLETE — Authentication, database schema, sidebar, topbar, login page, and all route placeholders are ready. Run `docker-compose up -d` then seed the DB. Proceed to Phase 2 for Employee Management + Dashboard."